**Requirement 2**

Design 1

Create class (mostly from assignment 1):

a. Interface:
    i. Consumable
b. Abstract class:
    i. Tree
    ii. Fruit
    iii. Animal
c. Concrete Class:
    i. Cave
    ii. Tundra
    iii. Meadow
    iv. HazelnutTree
    v. AppleTree
    vi. YewBerryTree
    vii. Hazelnut
    viii. Apple
    ix. YewBerry
    x. Bear
    xi. Deer
    xii. Wolf
    xiii. ConsumeAction
    xiv. ConsumeBehaviour
    xv. WanderBehaviour
d. Enum Class:
    i. Food

| Pro | Cons |
|---|---|
| Easier to read and understand | Duplication of code inside the cave, meadow, and Tundra |
| Direct implementation that each concrete class performs its own behaviour directly | Tight coupling, classes directly tied to specific animals, and have multiple if/else for different animal types |
| | Difficult to extend, need to modify each spawnable place class when needed to spawn new Animal |

Design 2

Create a new class/interface

a. Interface:
    i. Spawnable
    ii. Warmable

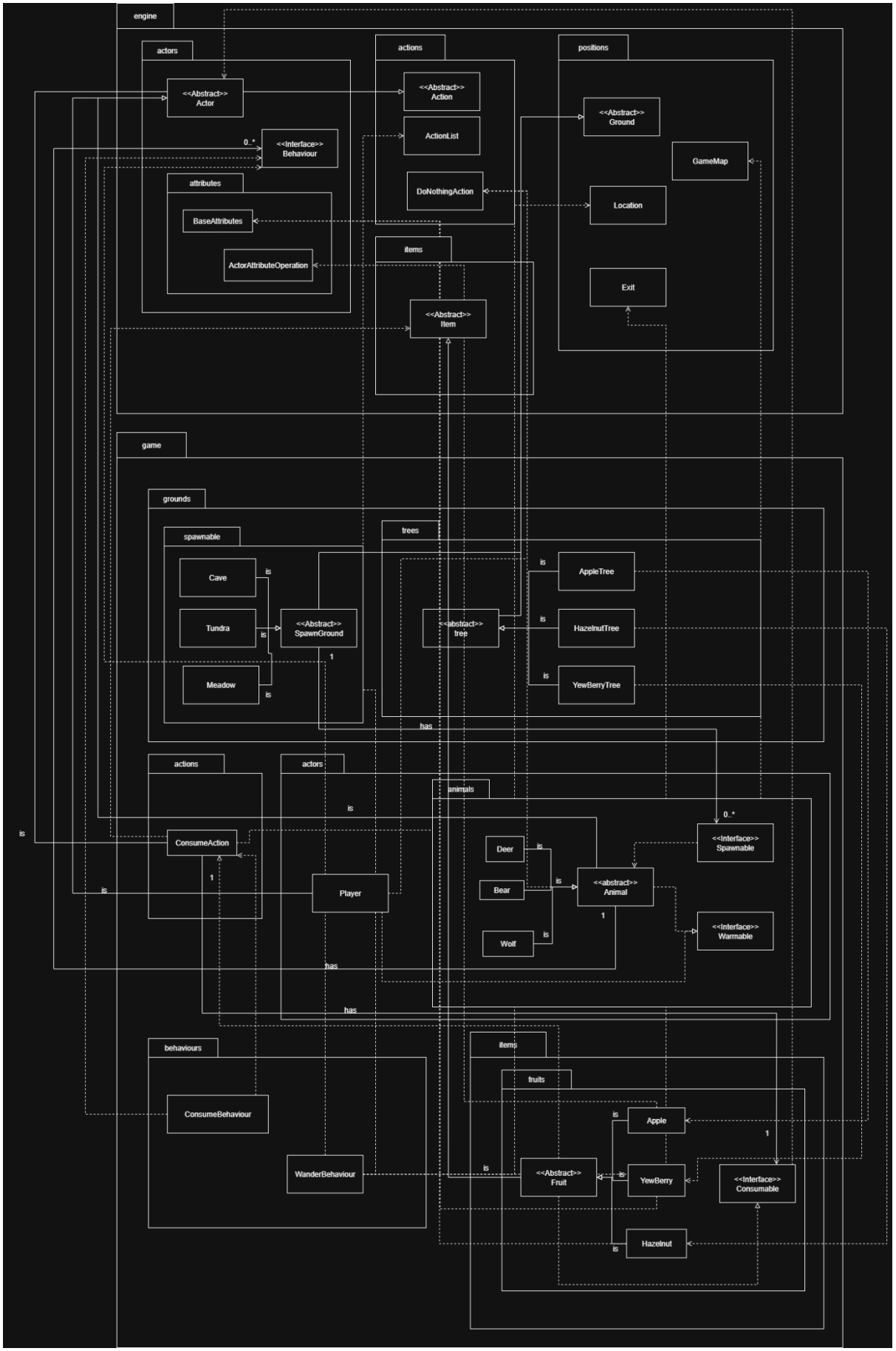
b. Abstract class:
    i. SpawnGround

| Pro | Cons |
|---|---|
| Encapsulation of spawn logic behind the Spawnable Interface, it can call create() without knowing what animal spawns | More complex for beginners |
| Better OOP principle by enhancing the open-closed principle, can add new animals without modifying the existing classes | Overhead for small projects |
| Follow the DRY principle, all spawn logic exists in the abstract SpawnGround class, hence no duplication in the Cave, Tundra, and Meadow classes | |
| Can be extended easily | |

| | |
|---|---|
| - Add new ground – extend SpawnGround<br>- Add new spawnable animal – implement Spawnable<br>- Add warmth conditions – implement Warmable | |

**Choose:**

We suggest design 2 as it was easier to extend, cleaner, and reusable compared to design 1. Design 1 is easier to implement and readable for a small project, but it was not suitable for a large project, as it may cause duplicate code and high coupling. Design 2 has a better OOP design and follows the DRY principle and aligning the SOLID principles. This ensures flexibility and scalability for future extensions without changes to the existing code.


engine
actors
<<Abstract>>
Actor
0..*
<<Interface>>
Behaviour
attributes
BaseAttributes
ActorAttributeOperation
actions
<<Abstract>>
Action
ActionList
DoNothingAction
items
<<Abstract>>
Item
positions
<<Abstract>>
Ground
GameMap
Location
Exit
game
grounds
spawnable
Cave
Tundra
Meadow
is
<<Abstract>>
SpawnGround
1
trees
<<abstract>>
tree
AppleTree
HazelnutTree
YewBerryTree
has
actions
ConsumeAction
1
actors
Player
animals
Deer
Bear
Wolf
<<abstract>>
Animal
1
0..*
<<Interface>>
Spawnable
<<Interface>>
Warmable
has
behaviours
ConsumeBehaviour
WanderBehaviour
items
fruits
<<Abstract>>
Fruit
Apple
YewBerry
Hazelnut
1
<<Interface>>
Consumable

2. What classes will exist in the extended system
    a. Interface:
        i. Spawnable
        ii. Warmable
        iii. Consumable
    b. Abstract class:
        i. SpawnGround
        ii. Tree
        iii. Fruit
        iv. Animal
    c. Concrete Class:
        i. Cave
        ii. Tundra
        iii. Meadow
        iv. HazelnutTree
        v. AppleTree
        vi. YewBerryTree
        vii. Hazelnut
        viii. Apple
        ix. YewBerry
        x. Bear
        xi. Deer
        xii. Wolf
        xiii. ConsumeAction
        xiv. ConsumeBehaviour
        xv. WanderBehaviour
    d. Enum Class:
        i. Food

3. Roles & Responsibilities
    a. Warmable interface defines warmth logic and represents any actor that can lose warmth and become cold
    b. Abstract animal class extends Actor implementing Warmable, which holds the shared logic including consume action, warmth, and movement
    c. Spawnable interface defines a functional interface for creating new Animal instances and ensuring spawn logic is decoupled from the spawner
    d. Abstract SpawnGround extends Grounds, holds spawn logic with a list of spawnable. It handles spawn animal turns, chance, and the specific conditions that occur for each spawnable place
        i. Tundra spawns an animal every turn with 5% chance and grants the spawned animals specific health and warmth conditions
        ii. Cave is a concrete class that extends SpawnGround that spawns animals every 5 turns, with each animal having an equal chance to spawn
        iii. Meadow spawn animal with its own rate and chance
    e. Consumable interface defines any item that can be consumed
    f. Abstract fruit class represents a consumable item that provides a consume logic and implements specific effects by overriding effects

4. How these classes relate to and interact with the existing system.
    a. SpawnGround extends from Ground, providing the ability for the environment to generate items or actors over time.
    b. Spawnable acts as a contract for items that can be spawned
    c. Each animal class doesn't implement the Spawnable interface, but Bear::new are function objects that plug into Spawnable which shows indirect dependency.
    d. Warmable provides a path for objects to interact with warmth logic
    e. Consumable allows the item to define what happens when it is consumed
    f. Fruit parent class extends from Item and implements Consumable, enabling each fruit class to be carried by actors and trigger ConsumeAction when used
    g. ConsumeAction extends from Action, allowing the actor to consume items from the grounds or the inventory

h. Consume Action and WanderAction integrates with the actor's behaviour that allows the actor to consume a consumable item and wander around on the map

i. Polymorphism ensures all classes are smoothly organised, any item that can be consumed implementing Consumable, any ground that can spawn an animal extending SpawnGround, any object or actor interacts with heat logic implementing Warmable

    i. This reduces coupling and follows the DRY principle

5. How the classes interact to deliver functionality.

    a. Actor interacts with items on the ground or inventory. If the item is consumable, they can consume it and perform the related effects, which trigger a ConsumeAction
    b. ConsumeAction calls the consume method from the item. It applies the effect of consumables, such as restoring health
    c. Fruit class extends Item and implements Consumable, defines its own specific effect
    d. ConsumeBehaviour generates ConsumeAction for the actor; it can automatically run the consume logic.
    e. SpawnGround checks in every tick to spawn the spawnable, such as animals. When spawn logic occurs, it calls the spawn method defined by the Spawnable item.
    f. Spawnable item provides details about how it was created and placed on the map. This allows animals to spawn without modifying the code in Ground
    g. Warmable interacts with any object or actor that performs warmth logic
    h. These systems are tied through polymorphism, which makes the functionalities reusable and easily extensible for future items